AMPLIACIÓN AL CÓDIGO ANTERIOR:

Ejercicio práctico — Ampliación del sistema de empleados

Situación

La empresa del ejercicio anterior quiere ampliar su aplicación para gestionar nuevos tipos de empleados.

Partiendo del proyecto proporcionado por el profesor, deberás añadir nuevas funcionalidades utilizando correctamente los conceptos de Programación Orientada a Objetos.

El programa debe seguir funcionando después de realizar todos los cambios.

---

1. Crear la clase Diseñador

Crea una nueva clase:

```
Diseñador
```

que herede de:

```
Empleado
```

La clase tendrá un único atributo adicional:

```
private String herramienta;
```

Por ejemplo:

- Photoshop
- Illustrator

Constructor

Crea un constructor que permita inicializar todos los datos necesarios.

Métodos

Implementa:

```
public void diseñar()
```

que muestre un mensaje indicando que el diseñador está trabajando.

Sobrescribe:

```
calcularSueldo()
```

El diseñador tendrá un suplemento de 200 € sobre su sueldo base.

También deberá sobrescribir:

```
trabajar()
```

y:

```
mostrarInformacion()
```

---

2. Ampliar la interfaz TrabajadorRemoto

Añade a la interfaz un nuevo método:

```
void realizarReunionOnline();
```

Todos los trabajadores que implementen la interfaz deberán proporcionar una implementación de este método.

Modifica Programador para implementar el nuevo método.

---

3. Crear ProgramadorSenior

Crea una nueva clase:

```
ProgramadorSenior
```

que herede de:

```
Programador
```

Añade el atributo:

```
private int añosExperiencia;
```

El constructor deberá permitir establecer todos los datos del programador senior.

Sobrescribe:

```
calcularSueldo()
```

Un programador senior tendrá:

```
sueldo base + 600 €
```

Además, sobrescribe:

```
trabajar()
```

para mostrar un mensaje diferente al de un programador normal.

Crea también:

```
public void revisarCodigo()
```

que muestre un mensaje indicando que el programador senior está revisando código.

---

4. Añadir sobrecarga

En Programador crea un nuevo método:

```
public void programar()
```

que ya existe.

Añade una versión sobrecargada:

```
public void programar(String proyecto)
```

Por ejemplo:

- Ana está programando en Java.
- Ana está programando el proyecto ERP en Java.

Comprueba desde Main que las dos versiones funcionan correctamente.

---

5. Modificar Empresa

La clase Empresa debe poder mostrar solamente los empleados que sean programadores.

Añade:

```
public void mostrarProgramadores()
```

El método deberá recorrer la lista de empleados y mostrar únicamente los objetos que sean Programador o alguna clase hija de Programador.

Por ejemplo, también deberá incluir:

```
Programador
ProgramadorSenior
```

Puedes utilizar:

```
instanceof
```

para realizar esta comprobación.

---

6. Añadir un método para buscar empleados

En Empresa, crea:

```
public Empleado buscarEmpleado(int id)
```

El método recorrerá la lista de empleados.

Si encuentra un empleado cuyo id coincida con el parámetro, devolverá dicho empleado.

Si no encuentra ninguno:

```
return null;
```

Desde Main, prueba ambos casos:

- Buscar un empleado que exista.
- Buscar un empleado que no exista.

---

7. Utilizar polimorfismo

Modifica el main para crear al menos:

- 2 Programador
- 1 ProgramadorSenior
- 2 Administrativo
- 1 Diseñador

Todos ellos deberán incorporarse a:

```
Empresa empresa
```

mediante:

```
empresa.contratar(...)
```

No debes crear diferentes listas para cada tipo.

Todos los objetos deberán almacenarse en:

```
ArrayList<Empleado>
```

---

8. Comprobar el polimorfismo

Ejecuta:

```
empresa.hacerTrabajarAEmpleados();
```

Comprueba que cada objeto ejecuta su propia versión de:

```
trabajar()
```

Explica posteriormente por escrito por qué ocurre esto.

---

9. Trabajadores remotos

Crea en Main una variable:

```
TrabajadorRemoto trabajador;
```

Haz que pueda almacenar un objeto Programador.

Posteriormente haz que almacene un objeto Diseñador.

Comprueba que puedes llamar a los métodos definidos en la interfaz.

---

10. Prueba final

El main deberá terminar mostrando información similar a:

```
====================================
      EMPRESA TECHSOLUTIONS
====================================

--- EMPLEADOS ---

PROGRAMADOR
Nombre: Ana
...
```

```
PROGRAMADOR SENIOR
Nombre: Pedro
...

ADMINISTRATIVO
Nombre: Laura
...

DISEÑADOR
Nombre: Marta
...

--- TRABAJADORES ---

Ana está desarrollando software.
Pedro está trabajando como programador senior.
Laura está realizando tareas administrativas.
Marta está diseñando una nueva interfaz.

--- PROGRAMADORES ---

Ana
Pedro
Carlos

--- BÚSQUEDA ---

Empleado encontrado: Pedro

Empleado no encontrado.
```

El formato exacto de los mensajes es libre.